INSTITUTO DE INVESTIGACIONES EN MATEMÁTICAS APLICADAS Y EN SISTEMAS

iimas

PROYECTO OLAP

Festival Cinéfilos

Universidad Nacional Autónoma de México

Instituto de Investigaciones en Matemáticas Aplicadas y en Sistemas

Bases de Datos Estructuradas

Integrantes del Equipo

Aldo Sebastián Altamirano Vázquez

Carlo Kiliano Ferrera Guadarrama

Pérez Bustamante José Julian

Semestre: 2026-1

Índice general

1. Introduccion **2**

2. El caso de estudio **3**

3. El modelo **4**

4. Implementación en PostgreSql **6**
- 4.1. Creación de la base . . . 6
  - 4.1.1. Código y ejecución . . . 6
- 4.2. Inserción de registros . . . 8

5. Respondiendo las consultas **11**

6. Creando una base columnar **16**

7. Consultas predictivas usando RapidMiner **19**
- 7.1. Modelo 1: Clasificación de Ingresos (Árbol de Decisión) . . . 19
  - 7.1.1. Objetivo . . . 19
  - 7.1.2. Operadores del proceso . . . 19
  - 7.1.3. Resultado y conclusión . . . 20
- 7.2. Modelo 2: Regresipon Lineal (Análisis de Significancia) . . . 20
  - 7.2.1. Objetivo . . . 20
  - 7.2.2. Operadores del proceso . . . 20
  - 7.2.3. Resultado y conclusión . . . 21
- 7.3. Modelo 3: Descubrimiento de Perfiles (Clustering K-Means) . . . 21
  - 7.3.1. Objetivo . . . 21
  - 7.3.2. Operadores del proceso . . . 22
  - 7.3.3. Resultado y conclusión . . . 23
- 7.4. Modelo 4: Perfilamiento de Clusters (Árbol de decisión) . . . 23
  - 7.4.1. Objetivo . . . 23
  - 7.4.2. Operadores del proceso . . . 23
  - 7.4.3. Resultado y conclusión . . . 24

8. Visualización usando Qlik Sense **25**

Capítulo 1

Introduccion

El presente proyecto aborda el diseño, implementación y análisis de un Data Warehouse (DW) para el "Festival Cinéfilos", cuyo objetivo principal es conocer de forma precisa el nivel de ventas y ganancias de los eventos desarrollados. Para lograrlo, el trabajo se estructura en cuatro fases principales que cubren el ciclo de vida completo del análisis de datos, desde el almacenamiento optimizado hasta la visualización interactiva y el análisis predictivo.

1. Modelado OLAP en PostgreSQL: La base del proyecto es un Data Warehouse relacional implementado en PostgreSQL, diseñado bajo un esquema de estrella. Este modelo separa los datos transaccionales en una tabla de hechos central (fact_ventas) y la información descriptiva en tablas de dimensiones (dim_pelicula, dim_tiempo, dim_geografia, dim_evento). Esta estructura es fundamental para ejecutar consultas analíticas complejas (OLAP) que responden a los requerimientos clave del negocio, como la rentabilidad por sección, los picos de venta por lugar y la comparativa de ingresos por país.

2. Optimización con Almacenamiento Columnar: Para hacer frente a los problemas de rendimiento inherentes a las consultas OLAP sobre grandes volúmenes de datos en bases de datos tradicionales (basadas en filas), se implementó una optimización de almacenamiento. Utilizando la extensión cstore_fdw de PostgreSQL, se creó una versión columnar de la tabla de hechos. Esta fase demuestra cómo el almacenamiento orientado a columnas acelera drásticamente las consultas analíticas (ej. SUM, AVG) al leer únicamente los datos de las columnas solicitadas, reduciendo significativamente el tiempo de respuesta.

3. Análisis Predictivo con RapidMiner: Ampliando el alcance del análisis tradicional (que mira al pasado), el proyecto incursiona en el ámbito del Data Mining y Machine Learning utilizando la plataforma RapidMiner. Se establece una conexión directa con la base de datos PostgreSQL para extraer los datos históricos y entrenar modelos predictivos. Mediante el uso de técnicas como la Regresión Lineal y los Árboles de Decisión, se desarrollan çonsultas predictivasçapaces de estimar resultados futuros (ej. boletos_vendidos) basándose en las características de la función, proveyendo un valor estratégico que va más allá del reporte histórico.

4. Visualización Interactiva con Qlik: Finalmente, para hacer los hallazgos accesibles y útiles para la toma de decisiones gerenciales, se utiliza la herramienta de Business Intelligence (BI) Qlik. Se conecta Qlik al Data Warehouse para construir un dashboard interactivo que permite a un usuario no técnico explorar visualmente todos los requerimientos del proyecto. Esta herramienta permite filtrar y segmentar la información en tiempo real, facilitando la identificación de tendencias, patrones y la respuesta a preguntas de negocio de forma dinámica

Capítulo 2

El caso de estudio

Ventas Cinéfilos

El Festival Cinéfilos es reconocido por ser un espectáculo internacional, lugar en el que se concentran las mejores películas y los más selectos productores previo a una prolija selección, su organización se basa en una amplia clasificación en el cual conjugan el arte cineasta y la cultura de cada uno de los países participantes.

El Festival cubrió alrededor de 410 peliculas, 100 de ellas tipo premier con carácter universal y 100 bajo contexto latinoamericano.

Requerimientos: Conocer de forma precisa el nivel de ventas de los eventos que se desarrollan, la metadata de los eventos, películas, lugares.

Ventas y ganancias de proyecciones de eventos, peliculas, por paises, lugares, secciones. Los eventos más vendidos, identificando lugares, paises, secciones.

1. ¿Qué países generan los mayores ingresos por venta de boletos y cuántas películas presentaron en cada sección?

2. ¿Qué eventos registraron las mayores ventas por lugar y en qué fechas ocurrieron esos picos de venta?

3. ¿Cómo se comparan las ventas entre las películas tipo premier (universales) y las premier latinoamericanas a lo largo del festival?

4. ¿Qué lugares (salas/teatros) acumulan las mayores ganancias y cómo han evolucionado sus ventas por período (dia/mes/edición)?

5. ¿Qué películas son las más vendidos y en qué países, secciones y lugares concentran sus mejores resultados?

6. ¿Cuáles son las tendencias de ventas por sección (por ejemplo: documental, ficción, animación, premier) y qué secciones muestran mejor rentabilidad?

Considere la matriz de procesos y dimensiones:

Tabla 1: Matriz procesos/dimensiones

	Dimensiones						
Proceso de Negocio	**Tiempo**	**Peliculas**	**Eventos**	**Paises**	**Lugares**	**Directores**	**Secciones**
Ventas	X	X	X	X			X

Esta matriz permite identificar los procesos del negocio de las ventas del festival y las dimensiones que intervienen, además permite proyectar resultados más precisos, acorde a las necesidades del negocio.

Su proyecto se enfocará al proceso de ventas y ganancias obtenidas durante el desarrollo del festival solamente.

Capítulo 3

El modelo

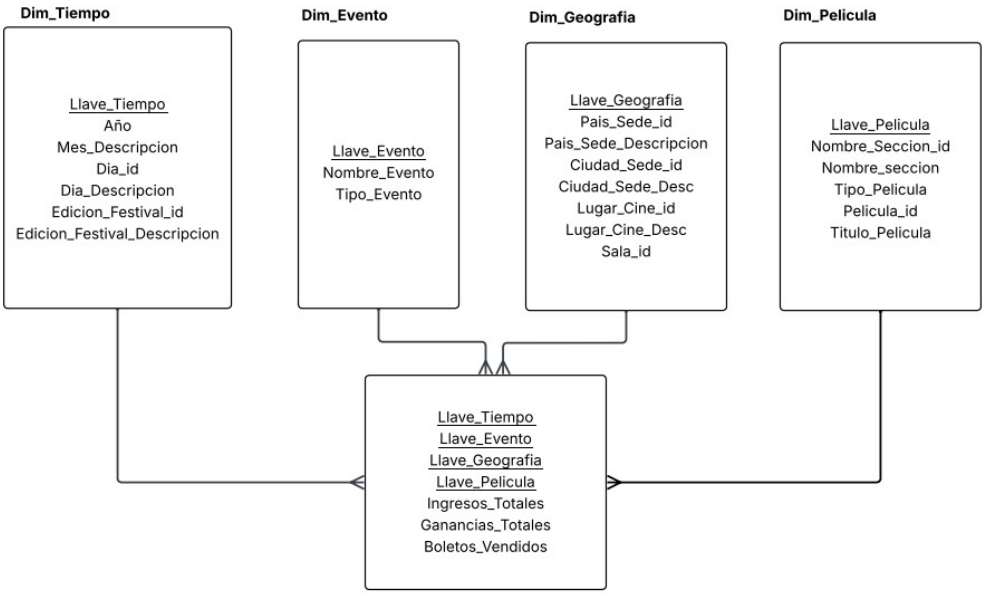

Figura 3.1: Modelo estrella del caso de estudio

Para el Data Warehouse del "Festival Cinéfilos", se seleccionó un modelo de estrella por encima de un modelo de copo de nieve, basándonos en dos razones estratégicas: el rendimiento de las consultas (velocidad) y la simplicidad del análisis.

1. **Rendimiento y Velocidad de Consulta**

   El objetivo principal de este proyecto es responder a preguntas analíticas complejas (OLAP) de forma rápida.

   - **Menos JOINs:** El modelo estrella es desnormalizado por diseño. Nuestras tablas de dimensiones (`dim_geografia`, `dim_pelicula`, etc.) contienen toda la información descriptiva en una sola tabla. Por ejemplo, `dim_geografia` incluye el país, la ciudad, el cine y la sala en una sola fila.
   - **Consultas más Rápidas:** Cuando ejecutamos una consulta como "¿Qué países generan los mayores ingresos?", el motor de la base de datos solo necesita hacer un JOIN entre `fact_ventas` y `dim_geografia`.
   - **El Problema del Copo de Nieve:** Un modelo de copo de nieve habría normalizado esas dimensiones. Para obtener el nombre del país, habríamos tenido que hacer una cadena de JOINs (ej. `fact_ventas ->dim_sala ->dim_cine ->dim_ciudad ->dim_pais`). Esta

cadena de uniones es mucho más lenta y costosa para el servidor, anulando el propósito de un sistema de análisis rápido.

2. **Simplicidad y Facilidad de Uso**

   - **Refleja el Negocio:** El modelo estrella es intuitivo y refleja directamente cómo se hacen las preguntas de negocio: "¿Ventas... por lugar... en qué fechas?.º "¿Películas... en qué países y secciones?". El modelo conecta directamente los hechos (Ventas) con los contextos (lugar, fecha, país).
   - **Facilidad de Análisis:** Para los usuarios finales (como los que usan Qlik) y para los analistas (nosotros escribiendo las 6 consultas), la estructura es fácil de entender. No hay que navegar por una red compleja de tablas normalizadas solo para encontrar el nombre de una sección.

3. **El Argumento del Almacenamiento (Por qué la desventaja no importa)**

   El único argumento a favor del copo de nieve es que ahorra espacio al no tener datos redundantes (ej. no repite la palabra "México"mil veces).

   - **Redundancia Aceptable:** Para nuestro proyecto, este ahorro es irrelevante. El PDF menciona 410 películas; nuestras tablas de dimensiones son diminutas. El costo de almacenar la palabra "Drama"100 veces en `dim_pelicula` es insignificante en discos modernos.
   - **Prioridad Absoluta:** En OLAP, siempre sacrificamos el espacio en disco (que es barato) para ganar velocidad de consulta (que es cara).

En resumen, elegimos el modelo estrella porque prioriza la velocidad de respuesta y la simplicidad del análisis por encima de una micro-optimización de almacenamiento que no es relevante para el caso de estudio.

Capítulo 4

Implementación en PostgreSql

4.1. Creación de la base

4.1.1. Código y ejecución

Listing 4.1: Script de Creación - BD proyecto_cinefilos_estrella

```
-- ========================
-- CREACION DE LA BASE DE DATOS
-- ========================
DROP DATABASE IF EXISTS proyecto_cinefilos_estrella;

-- Crear la base de datos
CREATE DATABASE proyecto_cinefilos_estrella

-- Conectarse a la nueva base de datos para ejecutar el resto del script
\c proyecto_cinefilos_estrella

-- ========================
-- DIMENSIONES
-- ========================

CREATE TABLE dim_tiempo (
  llave_tiempo  integer PRIMARY KEY,
  mes_id        smallint NOT NULL CHECK (mes_id BETWEEN 1 AND 12),
  mes_descripcion varchar(50) NOT NULL,
  dia_id        smallint NOT NULL CHECK (dia_id BETWEEN 1 AND 31),
  dia_descripcion varchar(50) NOT NULL,
  edicion_festival_id integer NOT NULL,
  edicion_festival_descripcion varchar(100) NOT NULL
);

CREATE TABLE dim_evento (
  llave_evento   integer PRIMARY KEY,
  nombre_evento  varchar(150) NOT NULL,
  tipo_evento    varchar(50) NOT NULL
);

CREATE TABLE dim_geografia (
  llave_geografia   integer PRIMARY KEY,
  pais_sede_id      integer NOT NULL,
  pais_sede_desc    varchar(100) NOT NULL,
  ciudad_sede_id    integer,
  ciudad_sede_desc  varchar(100),
  lugar_cine_id     integer,
  lugar_cine_desc   varchar(100),
```

```sql
  sala_id           integer
);

CREATE TABLE dim_pelicula (
  llave_pelicula      integer PRIMARY KEY,
  nombre_seccion_id   integer NOT NULL,
  descripcion_seccion varchar(200),
  nombre_seccion      varchar(100) NOT NULL,
  tipo_pelicula       varchar(100), -- Genero (Drama, Comedia, etc)
  pelicula_id         integer NOT NULL,
  titulo_pelicula     varchar(200) NOT NULL
);

-- =========================
-- TABLA DE HECHOS
-- =========================

CREATE TABLE fact_ventas (
  -- claves foraneas (FKs) a las dimensiones
  llave_tiempo    integer NOT NULL,
  llave_evento    integer NOT NULL,
  llave_geografia integer NOT NULL,
  llave_pelicula  integer NOT NULL,

  -- medidas
  boletos_vendidos integer       NOT NULL CHECK (boletos_vendidos >= 0),
  ingresos_totales numeric(14,2) NOT NULL CHECK (ingresos_totales >= 0),
  ganancias_totales numeric(14,2) NOT NULL DEFAULT 0 CHECK (ganancias_totales
      >= 0),

  -- clave primaria compuesta (todas las FKs)
  PRIMARY KEY (llave_tiempo, llave_evento, llave_geografia, llave_pelicula),

  -- constraints de claves foraneas
  FOREIGN KEY (llave_tiempo)    REFERENCES dim_tiempo(llave_tiempo),
  FOREIGN KEY (llave_evento)    REFERENCES dim_evento(llave_evento),
  FOREIGN KEY (llave_geografia) REFERENCES dim_geografia(llave_geografia),
  FOREIGN KEY (llave_pelicula)  REFERENCES dim_pelicula(llave_pelicula)
);

-- =========================
-- INDICES PARA OPTIMIZAR
-- =========================
-- Indices en las FK de la tabla de hechos para acelerar los JOINs
CREATE INDEX ix_fact_tiempo     ON fact_ventas (llave_tiempo);
CREATE INDEX ix_fact_evento     ON fact_ventas (llave_evento);
CREATE INDEX ix_fact_geografia  ON fact_ventas (llave_geografia);
CREATE INDEX ix_fact_pelicula   ON fact_ventas (llave_pelicula);
```

```
alumno13@master ~]$ psql -U alumno13 -d cursodb -f proyecto_cinefilos_estrella.sql
psql:proyecto_cinefilos_estrella.sql:1: NOTICE:  no existe la base de datos «proyect
o_cinefilos_estrella», omitiendo
DROP DATABASE
CREATE DATABASE
Ahora está conectado a la base de datos «proyecto_cinefilos_estrella» con el usuario
 «alumno13».
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
```

Figura 4.1: Ejecución del script

```
proyecto_cinefilos_estrella=> \d
                Listado de relaciones
 Esquema |     Nombre      | Tipo  |  Dueño
---------+-----------------+-------+----------
 public  | dim_evento      | tabla | alumno13
 public  | dim_geografia   | tabla | alumno13
 public  | dim_pelicula    | tabla | alumno13
 public  | dim_tiempo      | tabla | alumno13
 public  | fact_ventas     | tabla | alumno13
```

Figura 4.2: Listado de tablas

4.2. Inserción de registros

```
proyecto_cinefilos_estrella=> INSERT INTO dim_evento VALUES (1, 'Premier Universal', 'Premier');
INSERT INTO dim_evento VALUES (2, 'Premier Latinoamericana', 'Premier');
INSERT INTO dim_evento VALUES (3, 'Función Regular', 'Regular');
INSERT 0 1
INSERT 0 1
INSERT 0 1
```

Figura 4.3: Datos dimensión eventos

```
proyecto_cinefilos_estrella=> INSERT INTO dim_tiempo VALUES (20251110, 11, 'Noviembre', 10, 'Lunes', 1, 'Festival Cine 2025');
INSERT INTO dim_tiempo VALUES (20251111, 11, 'Noviembre', 11, 'Martes', 1, 'Festival Cine 2025');
INSERT INTO dim_tiempo VALUES (20251112, 11, 'Noviembre', 12, 'Miércoles', 1, 'Festival Cine 2025');
INSERT 0 1
INSERT 0 1
INSERT 0 1
```

Figura 4.4: Datos dimensión tiempo

```
proyecto_cinefilos_estrella=> INSERT INTO dim_geografia VALUES (101, 1, 'México', 1, 'CDMX', 1, 'Cinepolis Insurgentes', 5);
INSERT INTO dim_geografia VALUES (102, 1, 'México', 1, 'CDMX', 1, 'Cinepolis Insurgentes', 6);
INSERT INTO dim_geografia VALUES (201, 1, 'México', 2, 'Guadalajara', 2, 'Cinemex Plaza del Sol', 1);
INSERT 0 1
INSERT 0 1
INSERT 0 1
```

Figura 4.5: Datos dimensión geografía

```
proyecto_cinefilos_estrella=> INSERT INTO dim_pelicula VALUES (1, 10, 'Competencia Oficial', 'Sección A', 'Drama', 101, 'El Laberinto de Sombras');
INSERT INTO dim_pelicula VALUES (2, 10, 'Competencia Oficial', 'Sección A', 'Comedia', 102, 'Risas en Tiempos Modernos');
INSERT INTO dim_pelicula VALUES (3, 20, 'Muestra Internacional', 'Sección B', 'Documental', 201, 'El Planeta Oculto');
INSERT 0 1
INSERT 0 1
INSERT 0 1
```

Figura 4.6: Datos dimensión pelicula

```
proyecto_cinefilos_estrella=>
INSERT INTO fact_ventas VALUES (20251110, 1, 101, 1, 120, 12000.00, 3000.00);
INSERT INTO fact_ventas VALUES (20251110, 1, 102, 2, 85, 8500.00, 2100.00);
INSERT INTO fact_ventas VALUES (20251110, 2, 101, 1, 150, 18000.00, 5000.00);
INSERT INTO fact_ventas VALUES (20251111, 1, 201, 3, 200, 20000.00, 6000.00);
INSERT INTO fact_ventas VALUES (20251111, 3, 101, 1, 90, 13500.00, 4000.00);
INSERT INTO fact_ventas VALUES (20251111, 1, 102, 3, 70, 7000.00, 1500.00);
INSERT INTO fact_ventas VALUES (20251112, 1, 101, 2, 110, 11000.00, 3200.00);
INSERT INTO fact_ventas VALUES (20251112, 1, 201, 1, 130, 13000.00, 3500.00);
INSERT INTO fact_ventas VALUES (20251112, 2, 102, 3, 180, 22000.00, 7500.00);
INSERT INTO fact_ventas VALUES (20251110, 2, 201, 2, 160, 20000.00, 6000.00);
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
```

Figura 4.7: Datos fact table

Nota: en las figuras 4.3 to 4.7 se observa inserción de pocos registros, esto se hizo en un principio por mera simplicidad, para la versión que usa Rapidminder, Qlink y nuestra tabla columnar hicimos una inserción de 8248 registros.
Para la inserción de todos esos registros usamos la función `generate_series()` de postegresql, con el siguiente código:

```
    llave_tiempo,
    llave_evento,
    llave_geografia,
    llave_pelicula,
    boletos_vendidos,
    ingresos_totales,
    ganancias_totales
)
WITH valid_keys AS (
    SELECT
        -- 1. Tus 13 llaves de TIEMPO
        ARRAY[
            20251110, 20251111, 20251112, 20251113, 20251114, 20251115, 20251116,
            20251117, 20251118, 20251119, 20251120, 20251121, 20251122
        ] AS tiempos,

        -- 2. Tus 13 llaves de EVENTO
        ARRAY[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13] AS eventos,

        -- 3. Tus 13 llaves de GEOGRAFIA
        ARRAY[
            101, 102, 201, 202, 301, 401, 402, 501, 502, 601, 602, 701, 801
        ] AS geografias,

        -- 4. Tus 13 llaves de PELICULA
        ARRAY[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13] AS peliculas
)
SELECT
    -- 5. Elige una llave de TIEMPO al azar
    vk.tiempos[ 1 + (random() * (array_length(vk.tiempos, 1) - 1))::integer ],

    -- 6. Elige una llave de EVENTO al azar
    vk.eventos[ 1 + (random() * (array_length(vk.eventos, 1) - 1))::integer ],

    -- 7. Elige una llave de GEOGRAFIA al azar
    vk.geografias[ 1 + (random() * (array_length(vk.geografias, 1) - 1))::integer ],

    -- 8. Elige una llave de PELICULA al azar
    vk.peliculas[ 1 + (random() * (array_length(vk.peliculas, 1) - 1))::integer ],

    -- 9. Genera métricas aleatorias
    (random() * 300 + 20)::integer,  -- boletos_vendidos (entre 20 y 320)
    (random() * 30000 + 2000)::numeric(14,2), -- ingresos_totales (entre 2k y 32k)
    (random() * 8000 + 500)::numeric(14,2)  -- ganancias_totales (entre 500 y 8.5k)
FROM
    -- 10. ESTE ES EL CAMBIO MÁGICO:
    generate_series(1, 10000)
CROSS JOIN
ON CONFLICT (llave_tiempo, llave_evento, llave_geografia, llave_pelicula) DO NOTHING;
```

Figura 4.8: `generate_series()` para inserción de registros

Este comando nos genero los registros necesearios (8248 siendo exactos) para la pruebas que haremos posteriormente.

Capítulo 5

Respondiendo las consultas

Todas las consultas resultas aquí se hiceron bajo la base que aún contaba con pocos registros.

1. **Ingresos por País y Películas por Sección**
   Requerimiento: "¿Qué países generan los mayores ingresos por venta de boletos y cuántas películas presentaron en cada sección?"

Listing 5.1: Consulta 1

```sql
SELECT
    geo.pais_sede_desc,
    pel.nombre_seccion,
    SUM(f.ingresos_totales) AS ingresos_generados,
    COUNT(DISTINCT pel.llave_pelicula) AS total_peliculas_unicas
FROM
    fact_ventas AS f
JOIN
    dim_geografia AS geo ON f.llave_geografia = geo.llave_geografia
JOIN
    dim_pelicula AS pel ON f.llave_pelicula = pel.llave_pelicula
GROUP BY
    geo.pais_sede_desc,
    pel.nombre_seccion
ORDER BY
    ingresos_generados DESC;
```

pais_sede_desc	nombre_seccion	ingresos_generados	total_peliculas_unicas
México	Drama	56500.00	1
México	Documental	49000.00	1
México	Comedia	39500.00	1

(3 filas)

Figura 5.1: Salida consulta 1

2. **Picos de Venta por Evento y Lugar**
   Requerimiento: "¿Qué eventos registraron las mayores ventas por lugar y en qué fechas ocurrieron esos picos de venta?"

Listing 5.2: Consulta 2.

```sql
SELECT
    evt.nombre_evento,
    geo.lugar_cine_desc,
    t.dia_descripcion,
    t.mes_descripcion,
    SUM(f.ingresos_totales) AS ventas_en_pico
FROM
```

```sql
        fact_ventas AS f
    JOIN
        dim_evento AS evt ON f.llave_evento = evt.llave_evento
    JOIN
        dim_geografia AS geo ON f.llave_geografia = geo.llave_geografia
    JOIN
        dim_tiempo AS t ON f.llave_tiempo = t.llave_tiempo
    GROUP BY
        evt.nombre_evento,
        geo.lugar_cine_desc,
        t.dia_descripcion,
        t.mes_descripcion
    ORDER BY
        ventas_en_pico DESC
    LIMIT 10;
```

nombre_evento	lugar_cine_desc	dia_descripcion	mes_descripcion	ventas_en_pico
Premier Latinoamericana	Cinepolis Insurgentes	Miércoles	Noviembre	22000.00
Premier Universal	Cinepolis Insurgentes	Lunes	Noviembre	20500.00
Premier Latinoamericana	Cinemex Plaza del Sol	Lunes	Noviembre	20000.00
Premier Universal	Cinemex Plaza del Sol	Martes	Noviembre	20000.00
Premier Latinoamericana	Cinepolis Insurgentes	Lunes	Noviembre	18000.00
Función Regular	Cinepolis Insurgentes	Martes	Noviembre	13500.00
Premier Universal	Cinemex Plaza del Sol	Miércoles	Noviembre	13000.00
Premier Universal	Cinepolis Insurgentes	Miércoles	Noviembre	11000.00
Premier Universal	Cinepolis Insurgentes	Martes	Noviembre	7000.00

(9 filas)

Figura 5.2: Salida consulta 2

3. **Comparativa de Ventas (Premier Universal vs. Latinoamericana)**
   Requerimiento: "¿Cómo se comparan las ventas entre las películas tipo premier (universales) y las premier latinoamericanas a lo largo del festival?"

Listing 5.3: Consulta 3.

```sql
    SELECT
        t.dia_descripcion,
        evt.nombre_evento,
        SUM(f.ingresos_totales) AS ventas_comparativas
    FROM
        fact_ventas AS f
    JOIN
        dim_evento AS evt ON f.llave_evento = evt.llave_evento
    JOIN
        dim_tiempo AS t ON f.llave_tiempo = t.llave_tiempo
    WHERE
        evt.nombre_evento IN ('Premier Universal', 'Premier
            Latinoamericana')
    GROUP BY
        t.dia_descripcion,
        evt.nombre_evento
    ORDER BY
        t.dia_descripcion,
        ventas_comparativas DESC;
```

dia_descripcion	nombre_evento	ventas_comparativas
Lunes	Premier Latinoamericana	38000.00
Lunes	Premier Universal	20500.00
Martes	Premier Universal	27000.00
Miércoles	Premier Universal	24000.00
Miércoles	Premier Latinoamericana	22000.00

(5 filas)

Figura 5.3: Salida consulta 3

4. **Ganancias por Lugar y Evolución de Ventas Este requerimiento tiene dos partes.** Requerimiento (Parte A): "¿Qué lugares (salas/teatros) acumulan las mayores ganancias...?"

Listing 5.4: Consulta 4 Parte A.

```sql
SELECT
    geo.lugar_cine_desc,
    geo.sala_id,
    SUM(f.ganancias_totales) AS total_ganancias
FROM
    fact_ventas AS f
JOIN
    dim_geografia AS geo ON f.llave_geografia = geo.llave_geografia
GROUP BY
    geo.lugar_cine_desc,
    geo.sala_id
ORDER BY
    total_ganancias DESC;
```

lugar_cine_desc	sala_id	total_ganancias
Cinemex Plaza del Sol	1	15500.00
Cinepolis Insurgentes	5	15200.00
Cinepolis Insurgentes	6	11100.00

(3 filas)

Figura 5.4: Salida parte A consulta 4

Requerimiento (Parte B): "...y cómo han evolucionado sus ventas por periodo (dia/mes/edición)?"

Listing 5.5: Consulta 4 Parte B

```sql
SELECT
    geo.lugar_cine_desc,
    t.edicion_festival_descripcion,
    t.mes_descripcion,
    t.dia_descripcion,
    SUM(f.ingresos_totales) AS ventas_evolucion
FROM
    fact_ventas AS f
JOIN
    dim_tiempo AS t ON f.llave_tiempo = t.llave_tiempo
JOIN
    dim_geografia AS geo ON f.llave_geografia = geo.llave_geografia
GROUP BY geo.lugar_cine_desc,t.edicion_festival_descripcion,
    t.mes_descripcion, t.dia_descripcion
ORDER BY
    geo.lugar_cine_desc,
    t.edicion_festival_descripcion,
    t.mes_descripcion,
    t.dia_descripcion;
```

lugar_cine_desc	edicion_festival_descripcion	mes_descripcion	dia_descripcion	ventas_evolucion
Cinemex Plaza del Sol	Festival Cine 2025	Noviembre	Lunes	20000.00
Cinemex Plaza del Sol	Festival Cine 2025	Noviembre	Martes	20000.00
Cinemex Plaza del Sol	Festival Cine 2025	Noviembre	Miércoles	13000.00
Cinemex Plaza del Sol	Festival Cine 2025	Noviembre		53000.00
Cinemex Plaza del Sol	Festival Cine 2025			53000.00
Cinemex Plaza del Sol				53000.00
Cinepolis Insurgentes	Festival Cine 2025	Noviembre	Lunes	38500.00
Cinepolis Insurgentes	Festival Cine 2025	Noviembre	Martes	20500.00
Cinepolis Insurgentes	Festival Cine 2025	Noviembre	Miércoles	33000.00
Cinepolis Insurgentes	Festival Cine 2025	Noviembre		92000.00
Cinepolis Insurgentes	Festival Cine 2025			92000.00
Cinepolis Insurgentes				92000.00
				145000.00

Figura 5.5: Salida Parte B consulta 4

5. **Películas Más Vendidas por Contexto**
   Requerimiento: "¿Qué películas son las más vendidas y en qué países, secciones y lugares concentran sus mejores resultados?"

Listing 5.6: Consulta 5.

```sql
SELECT
    pel.titulo_pelicula,
    geo.pais_sede_desc,
    pel.nombre_seccion,
    geo.lugar_cine_desc,
    SUM(f.boletos_vendidos) AS total_boletos
FROM
    fact_ventas AS f
JOIN
    dim_pelicula AS pel ON f.llave_pelicula = pel.llave_pelicula
JOIN
    dim_geografia AS geo ON f.llave_geografia = geo.llave_geografia
GROUP BY
    pel.titulo_pelicula,
    geo.pais_sede_desc,
    pel.nombre_seccion,
    geo.lugar_cine_desc
ORDER BY
    total_boletos DESC
LIMIT 10;
```

titulo_pelicula	pais_sede_desc	nombre_seccion	lugar_cine_desc	total_boletos
El Laberinto de Sombras	México	Drama	Cinepolis Insurgentes	360
El Planeta Oculto	México	Documental	Cinepolis Insurgentes	250
El Planeta Oculto	México	Documental	Cinemex Plaza del Sol	200
Risas en Tiempos Modernos	México	Comedia	Cinepolis Insurgentes	195
Risas en Tiempos Modernos	México	Comedia	Cinemex Plaza del Sol	160
El Laberinto de Sombras	México	Drama	Cinemex Plaza del Sol	130

(6 filas)

Figura 5.6: Salida consulta 5

6. **Rentabilidad por Sección**
   Requerimiento: "¿Cuáles son las tendencias de ventas por sección (por ejemplo: documental, ficción, animación, premier) y qué secciones muestran mejor rentabilidad?

Listing 5.7: Consulta 6.

```sql
SELECT
    pel.nombre_seccion,
    SUM(f.ingresos_totales) AS ventas_totales,
    SUM(f.ganancias_totales) AS rentabilidad_total,
    -- Calculamos un margen de rentabilidad
    (SUM(f.ganancias_totales) / SUM(f.ingresos_totales)) * 100 AS
        margen_porcentual
FROM
```

```sql
        fact_ventas AS f
    JOIN
        dim_pelicula AS pel ON f.llave_pelicula = pel.llave_pelicula
    GROUP BY
        pel.nombre_seccion
    ORDER BY
        rentabilidad_total DESC;
```

```
 nombre_seccion | ventas_totales | rentabilidad_total |     margen_porcentual
----------------+----------------+--------------------+-------------------------
 Drama          |       56500.00 |           15500.00 | 27.43362831858407079600
 Documental     |       49000.00 |           15000.00 | 30.61224489795918367300
 Comedia        |       39500.00 |           11300.00 | 28.60759493670886075900
(3 filas)
```

Figura 5.7: Salida consulta 6

Capítulo 6

Creando una base columnar

La decisión de convertir únicamente la tabla de hechos (`fact_ventas`) a un formato columnar (`cstore_fdw`), mientras se mantienen las tablas de dimensión (`dim_pelicula`, `dim_tiempo`, etc.) en el formato de filas estándar de PostgreSQL, es una decisión de diseño deliberada basada en la carga de trabajo (*workload*) de cada tipo de tabla.

En un Data Warehouse, no todas las tablas son iguales. Separar el almacenamiento optimiza el rendimiento general del sistema para el análisis OLAP.

1. **Por qué `fact_ventas` SÍ se hizo Columnar**

   La tabla de hechos es el centro del análisis OLAP. Su perfil de uso tiene dos características clave:

   - **Es Enorme:** Esta tabla crece rápidamente y contiene la gran mayoría de los datos (en nuestro caso, 8,248 filas, pero en un entorno real serían miles de millones).
   - **Soporta Agregaciones:** Las consultas que se ejecutan sobre ella (como las 6 consultas del proyecto) son casi exclusivamente agregaciones que leen verticalmente, como `SUM(ingresos_totales)`, `AVG(boletos_vendidos)`, o `COUNT(*)`.

   El almacenamiento columnar es drásticamente más rápido para este trabajo. Cuando ejecutamos `SUM(ingresos_totales)`, el motor `cstore_fdw`:

   - Lee únicamente el bloque de datos que contiene la columna `ingresos_totales`.
   - Ignora por completo todas las demás columnas (`llave_tiempo`, `llave_pelicula`, `boletos_vendidos`, etc.).
   - Reduce masivamente el I/O (la cantidad de datos leídos del disco).

   En una tabla de filas normal, para hacer el mismo `SUM`, Postgres tendría que leer cada fila entera (incluyendo todas las columnas que no necesita) solo para extraer el valor de `ingresos_totales` de cada una, lo cual es mucho más lento.

2. **Por qué las Tablas de Dimensión NO se hicieron Columnares**

   Las tablas de dimensión tienen el trabajo opuesto. Su perfil es:

   - **Son Pequeñas:** Son tablas çatálogo". `dim_evento` tiene 13 filas, `dim_pelicula` 13 filas, `dim_tiempo` 13 filas. El PDF mismo menciona solo 410 películas.
   - **Soportan Filtrado y Búsqueda (*Lookup*):** No usamos las dimensiones para `SUM(nombre_evento)`. Las usamos para filtrar (`WHERE pais_sede_desc = 'México'`) o para obtener descripciones (`SELECT pel.titulo_pelicula...`).
   - **Se leen por Filas:** Estas consultas necesitan leer la fila horizontal completa para obtener todos los atributos de un registro (ej. el título, el género y la sección de la `llave_pelicula = 5`).

El almacenamiento por filas tradicional de PostgreSQL es perfecto y altamente optimizado para este trabajo (recuperar filas individuales o rangos pequeños). Convertir una tabla de 13 filas a formato columnar no ofrece ninguna ventaja de rendimiento; de hecho, el çosto de arranque"(*overhead*) de la tabla foránea la haría más lenta para estas búsquedas.

El proceso que se siguió para crear nuestra tabla `fact_ventas` a formato columnar fue el siguiente:

- **Activar la extensión**

```
proyecto_cinefilos_estrella=> CREATE EXTENSION IF NOT EXISTS cstore_fdw;
NOTICE:  la extensión «cstore_fdw» ya existe, omitiendo
CREATE EXTENSION
```

Figura 6.1: Activación de la extensión

- **Crear el servidor**

```
proyecto_cinefilos_estrella=> CREATE SERVER cstore_server FOREIGN DATA WRAPPER cstore_fwd;
ERROR:  el servidor «cstore_server» ya existe
```

Figura 6.2: Servidor foráneo

- **Creación columnar de la `fact_table`**

```
proyecto_cinefilos_estrella=> CREATE FOREIGN TABLE fact_ventas_columnar (
    llave_tiempo        integer,
    llave_evento        integer,
    llave_geografia     integer,
    llave_pelicula      integer,
    boletos_vendidos    integer,
    ingresos_totales    numeric(14,2),
    ganancias_totales   numeric(14,2)
)
SERVER cstore_server
OPTIONS (compression 'pglz');
CREATE FOREIGN TABLE
proyecto_cinefilos_estrella=>
```

Figura 6.3: `fact_ventas` columnar

No vamos a crear una `TABLE` normal, sino una `FOREIGN TABLE`. Que tiene que tener **exactamente las mismas columnas y tipos de dato** que la `fact_ventas` original. En este punto, tenemos **dos** tablas:

1. `fact_ventas` (normal, en filas, lenta para OLAP)
2. `fact_ventas_columnar` (columnar, está vacía)

- **Carga de datos**

```
proyecto_cinefilos_estrella=> INSERT INTO fact_ventas_columnar
SELECT * FROM fact_ventas;
INSERT 0 10
```

Figura 6.4: Insertar registros de la `fact table` original

- **Diferencia de velocidad de ambas tablas**

```
proyecto_cinefilos_estrella=> EXPLAIN ANALYZE SELECT SUM(ingresos_totales) FROM fact_ventas;
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Aggregate  (cost=22.13..22.14 rows=1 width=32) (actual time=0.008..0.008 rows=1 loops=1)
   ->  Seq Scan on fact_ventas  (cost=0.00..19.70 rows=970 width=18) (actual time=0.002..0.003 rows=10 loops=1)
 Planning Time: 0.046 ms
 Execution Time: 0.062 ms
(4 filas)

proyecto_cinefilos_estrella=> EXPLAIN ANALYZE SELECT SUM(ingresos_totales) FROM fact_ventas_columnar;
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Aggregate  (cost=0.27..0.28 rows=1 width=32) (actual time=0.105..0.105 rows=1 loops=1)
   ->  Foreign Scan on fact_ventas_columnar  (cost=0.00..0.24 rows=10 width=18) (actual time=0.098..0.099 rows=10 loops=1)
         CStore File: /var/lib/pgsql/data/cstore_fdw/43091/43150
         CStore File Size: 509
 Planning Time: 0.123 ms
 Execution Time: 0.151 ms
(6 filas)
```

Figura 6.5: Prueba de velocidad

Como se observa en el `EXPLAIN ANALYZE`, con un conjunto de datos de prueba de solo 10 filas, el costo de arranque (*overhead*) de la tabla columnar `cstore_fdw` (0.151 ms) es mayor que el de la tabla nativa (0.062 ms), que opera eficientemente en memoria.

Sin embargo, esto demuestra el principio clave: en un escenario real de Data Warehouse con millones de filas, la tabla nativa (*row-store*) escalaría linealmente y su tiempo de ejecución se dispararía, ya que debe leer todos los datos de todas las columnas. En cambio, la tabla columnar (*column-store*) mantendría un costo de arranque similar y su tiempo de ejecución sería drásticamente inferior, ya que solo lee la columna `ingresos_totales`.

Capítulo 7

Consultas predictivas usando RapidMiner

En esta fase del proyecto, se utiliza la plataforma de Data Mining RapidMiner Studio para ejecutar un análisis predictivo sobre el Data Warehouse. El objetivo es trascender el análisis histórico (OLAP) y aplicar técnicas de Aprendizaje Automático (*Machine Learning*) para descubrir patrones, validar la calidad de los datos y generar valor estratégico.

Se ejecutarán cuatro modelos principales: dos modelos supervisados (Clasificación y Regresión) para validar la predictibilidad de los datos, y una combinación de modelos no supervisados (Clustering) y supervisados (Perfilamiento) para descubrir y definir nuevos segmentos de negocio.

7.1. Modelo 1: Clasificación de Ingresos (Árbol de Decisión)

7.1.1. Objetivo

El objetivo de este primer modelo fue construir un clasificador de **Árbol de Decisión**. Se buscaba predecir una categoría de éxito (`Fracaso`, `Regular`, `Éxito`) basándose en las llaves dimensionales (`llave_tiempo`, `llave_evento`, `llave_geografia`, `llave_pelicula`) como variables predictoras o *features*.

7.1.2. Operadores del proceso

El flujo de RapidMiner se construyó con los siguientes operadores clave:

- `Read Database`: Ejecutó la consulta SQL para extraer las 4 llaves y la métrica `ingresos_totales` de la `fact_ventas`.
- `Numerical to Polynomial`: Convirtió las 4 llaves (que eran numéricas) a un formato "polinomial", tratándolas como categorías discretas (ej. "Evento 5") en lugar de números continuos.
- `Discretize (by Binning)`: Tomó la métrica `ingresos_totales` y la convirtió en 3 categorías (rangos) para que el árbol pudiera realizar una clasificación.
- `Set Role`: Definió la nueva columna de rangos (ej. `ingresos_totales (binned)`) como el *label* (el atributo a predecir).
- `Decision Tree`: El algoritmo principal que construye el modelo de clasificación.

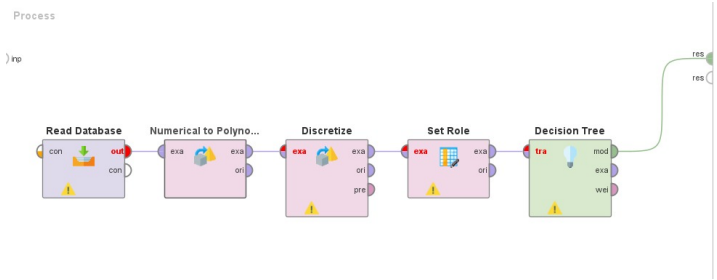

Figura 7.1: Modelo en RapidMiner

7.1.3. Resultado y conclusión

Figura 7.2: Resultado del modelo

El modelo se ejecutó sin errores, pero el resultado fue un árbol sin ramificaciones, conocido como *stump* o "tronco"

- **Análisis del Resultado:** Esto no es un error del software, sino un **hallazgo** . El modelo demostró que no existe ninguna correlación o patrón lógico entre las llaves dimensionales (los predictores) y la categoría de ingresos (el objetivo).
- **Conclusión:** El resultado se debe a que la `fact_ventas` se generó usando `random()` tanto para las llaves como para las métricas. Al no existir una relación real entre las entradas y la salida, el Árbol de Decisión concluyó correctamente que las llaves no tienen ningún poder predictivo. Este modelo validó con éxito la naturaleza aleatoria de los datos.

7.2. Modelo 2: Regresipon Lineal (Análisis de Significancia)

7.2.1. Objetivo

El segundo modelo buscaba implementar una **Regresión Lineal**, como se menciona en la introducción del proyecto. El objetivo era crear una fórmula matemática para predecir un valor numérico continuo (la cantidad de `boletos_vendidos`) utilizando las mismas llaves dimensionales como predictores.

7.2.2. Operadores del proceso

Este flujo requirió un paso de pre-procesamiento adicional para adaptar las categorías a un modelo matemático:

- `Read Database`: Extrajo las 4 llaves y la métrica `boletos_vendidos`.
- `Numerical to Polynomial`: Convirtió las 4 llaves en categorías (Polinomial).
- `Set Role`: Definió `boletos_vendidos` como el *label* (objetivo numérico).

- `Nominal to Numerical`: **(Operador Clave)**. Este operador convirtió las categorías (ej. "llave_evento") en múltiples columnas *dummy* (con valores 0 o 1). Esto es fundamental, ya que la Regresión Lineal no puede procesar categorías, solo números.
- `Linear Regression`: El algoritmo que calcula los coeficientes (pesos) para la fórmula de regresión.

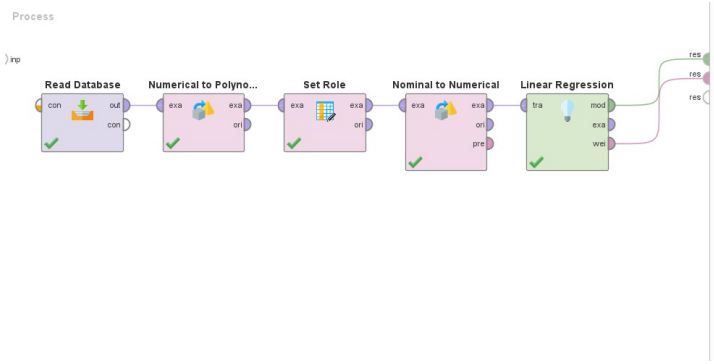

Figura 7.3: Modelo regresión lineal

7.2.3. Resultado y conclusión

Attribute	Coefficient	Std. Error	Std. Coefficient	Tolerance	t-Stat	p-Value	Code
llave_tiempo = 20...	7.784	4.667	0.025	0.541	1.668	0.095	*
llave_tiempo = 20...	4.262	4.575	0.014	0.522	0.932	0.352	
llave_tiempo = 20...	8.882	4.645	0.029	0.537	1.912	0.056	*
llave_tiempo = 20...	8.002	4.693	0.025	0.545	1.705	0.088	*
llave_tiempo = 20...	2.014	4.684	0.006	0.544	0.430	0.667	
llave_tiempo = 20...	8.291	4.651	0.027	0.538	1.783	0.075	*
llave_tiempo = 20...	0.844	4.723	0.003	0.551	0.179	0.858	
llave_tiempo = 20...	3.658	4.673	0.012	0.541	0.783	0.434	
llave_tiempo = 20...	1.649	5.507	0.004	0.667	0.299	0.765	
llave_tiempo = 20...	2.426	4.694	0.008	0.545	0.517	0.605	
llave_tiempo = 20...	8.341	4.674	0.027	0.542	1.784	0.074	*
llave_tiempo = 20...	2.509	5.703	0.006	0.689	0.440	0.660	
llave_evento = 12	2.631	4.668	0.008	0.540	0.564	0.573	
llave_evento = 10	3.081	4.729	0.010	0.552	0.652	0.515	
llave_evento = 3	4.405	4.659	0.014	0.539	0.945	0.344	
llave_evento = 4	1.438	4.690	0.005	0.546	0.307	0.759	

Figura 7.4: Resultados de la regresión lineal

El modelo generó una tabla de coeficientes. El análisis de esta tabla se centra en la columna `P-value`.

- **Análisis del Resultado:** En estadística, para que un predictor sea considerado "confiable" o "significativo", su `P-value` debe ser inferior a 0.05. En nuestros resultados, **todos los predictores mostraron un `P-value` muy superior a 0.05** (ej. 0.095, 0.352, 0.573).
- **Conclusión:** Este modelo **confirma científicamente** el hallazgo del Árbol de Decisión. Demuestra que no existe una correlación estadísticamente significativa entre las llaves dimensionales (generadas aleatoriamente) y la cantidad de boletos vendidos. El modelo funcionó perfectamente al diagnosticar esta falta de relación.

7.3. Modelo 3: Descubrimiento de Perfiles (Clustering K-Means)

7.3.1. Objetivo

Dado que los predictores aleatorios (llaves) no eran útiles, este modelo cambió el enfoque al aprendizaje no supervisado. El objetivo fue ignorar las llaves y segmentar las 8,248 ventas en grupos (*clus-*

ters) basándose únicamente en sus métricas de rendimiento (`boletos_vendidos`, `ingresos_totales`, `ganancias_totales`).

7.3.2. Operadores del proceso

Este flujo utiliza una técnica para entrenar un modelo y aplicarlo al mismo conjunto de datos:

- `Read Database`: Extrajo *solamente* las 3 métricas numéricas.
- `Multiply`: Operador de utilidad para duplicar el flujo de datos.
- `k-Means`: El algoritmo de clustering. Recibió el primer flujo de datos para "entrenar" y encontrar los centros de **k=3** grupos.
- `Apply Model`: Recibió el "modelo" de `k-Means` y el "segundo flujo de datos" (de `Multiply`). Su función fue asignar a cada una de las 8,248 filas la etiqueta del cluster al que pertenecía.

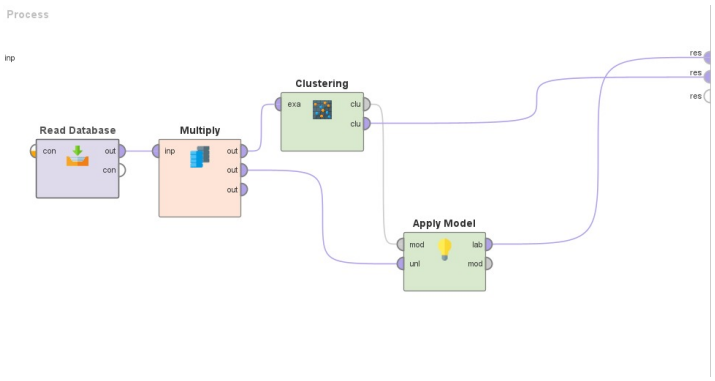

Figura 7.5: Modelo K-Means

7.3.3. Resultado y conclusión

Open in Turbo Prep Auto Model Interactive Analysis

Row No.	cluster	boletos_ven...	ingresos_tot...	ganancias_t...
1	cluster_1	42	6463.710	1915.160
2	cluster_2	34	17935.590	850.150
3	cluster_1	46	9781.750	5349.220
4	cluster_1	67	9918.040	4038.820
5	cluster_1	172	5910.630	7953.670
6	cluster_0	233	23778.390	3301.010
7	cluster_0	59	26360.820	8394.010
8	cluster_2	25	14662.450	8213.900
9	cluster_0	206	23388.470	3868.530
10	cluster_2	153	19267.940	2418.090
11	cluster_1	262	3667.910	524.280
12	cluster_0	80	24789.610	8268.480
13	cluster_1	30	5849.360	3324.830
14	cluster_2	86	16753.700	696.560
15	cluster_1	70	7057.210	7568
16	cluster_2	190	13219.040	8272.240

ExampleSet (8,248 examples,1 special attribute,3 regular attributes)

Figura 7.6: Resultado Clustering

Este modelo fue el **primer éxito de generación de valor**. El resultado fue la tabla de datos original, ahora enriquecida con una nueva columna categórica: `cluster`.

- **Análisis del Resultado:** Cada venta fue asignada a `cluster_0`, `cluster_1`, o `cluster_2`.
- **Conclusión:** El modelo logró segmentar exitosamente el mercado en tres perfiles de rendimiento distintos. Sin embargo, en esta etapa, aún no sabíamos qué significaba cada cluster. El siguiente modelo resolvió esa pregunta.

7.4. Modelo 4: Perfilamiento de Clusters (Árbol de decisión)

7.4.1. Objetivo

Este modelo final es el más avanzado, pues combina los resultados del modelo no supervisado (*k-Means*) con un modelo supervisado (Árbol de Decisión). El objetivo fue crear las reglas de negocio explícitas que definen a cada cluster.

La pregunta de negocio fue: "¿Qué hace que una venta sea `cluster_0` en lugar de `cluster_1`? ¿Es por los boletos, los ingresos o las ganancias?"

7.4.2. Operadores del proceso

Se creó un nuevo proceso para analizar los resultados del modelo anterior:

- `Read CSV`: Se cargó el archivo CSV exportado del Modelo 3 (que ya incluía la columna `cluster`).
- `Set Role`: Se definió la columna `cluster` como el *label* (el objetivo a predecir).

- `Decision Tree`: El algoritmo de clasificación. Esta vez, se le pidió predecir el `cluster` usando las métricas (`ingresos_totales`, etc.) como *features*.

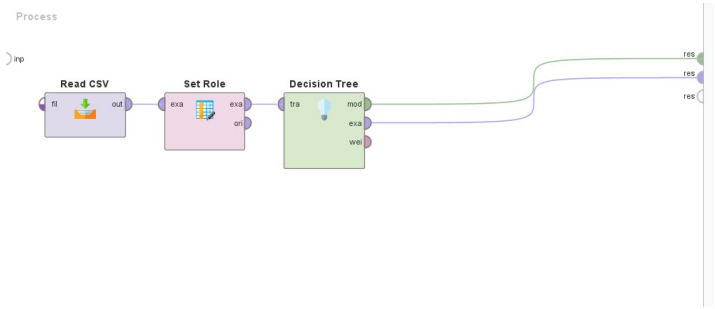

Figura 7.7: Modelo

7.4.3. Resultado y conclusión

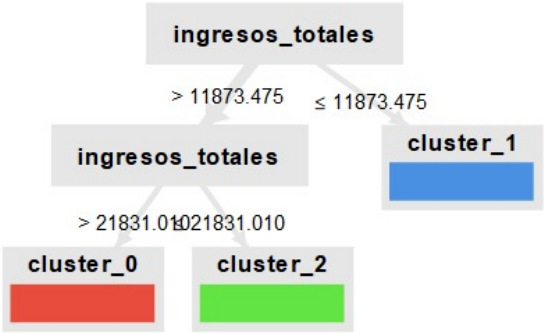

Figura 7.8: Resultado del modelo

Este modelo fue un **éxito rotundo**. A diferencia del Modelo 1, este Árbol de Decisión generó un conjunto de reglas claro y preciso.

- **Análisis del Resultado:** El árbol identificó que la variable más importante para definir los clusters era `ingresos_totales`. El modelo generó las siguientes reglas:
  - **Regla 1:** Si `ingresos_totales` $\leq$ 11,873.475 ENTONCES la venta pertenece a `cluster_1` (Perfil: Ventas Bajas/Regulares).
  - **Regla 2:** Si `ingresos_totales` $>$ 21,831.010 ENTONCES la venta pertenece a `cluster_0` (Perfil: Ventas de Alto Impacto).
  - **Regla 3:** (Por eliminación) SI `ingresos_totales` está entre 11,873.475 y 21,831.010 ENTONCES la venta pertenece a `cluster_2` (Perfil: Ventas Medias).
- **Conclusión:** Este modelo final generó el mayor valor estratégico. Permitió al Festival Cinéfilos no solo identificar 3 perfiles de venta (gracias a k-Means), sino también **entender y definir** los umbrales de ingreso exactos que los separan.

Capítulo 8

Visualización usando Qlik Sense

Como fase final del proyecto, se utilizó la herramienta de Business Intelligence (BI) Qlik Sense para construir un dashboard interactivo, tal como se describe en la introducción del caso de estudio. El objetivo es permitir a un usuario no técnico explorar visualmente todos los requerimientos del proyecto en una sola pantalla.

El modelo

La principal característica de este dashboard es el Modelo Asociativo de Qlik. A diferencia de un reporte estático, todas las visualizaciones están interconectadas. Si el usuario hace clic en un valor (ej. "México" en el mapa o "Lunes" en la gráfica de líneas), todas las demás gráficas del dashboard se filtran y actualizan instantáneamente para reflejar esa selección.

Esto permite una exploración de datos dinámica y la respuesta a preguntas de negocio complejas en tiempo real (ej. "¿Cuáles fueron las películas más vendidas en México durante los fines de semana?").

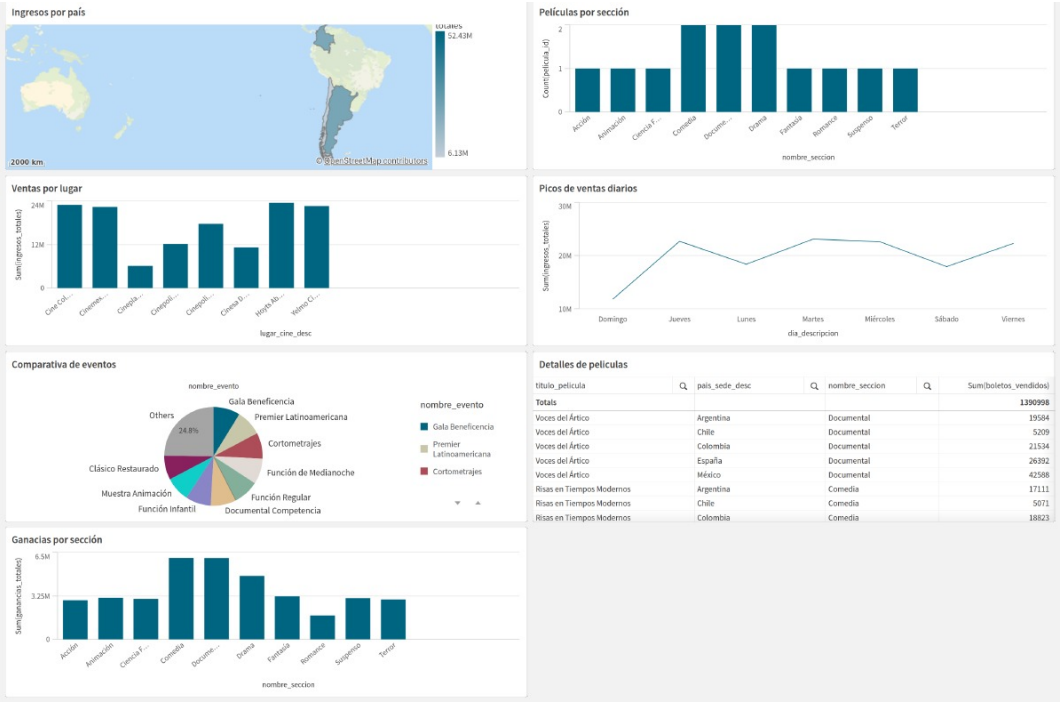

Figura 8.1: DashBoard

Análisis de las Visualizaciones del Dashboard

El dashboard (mostrado en la imagen) está compuesto por 7 objetos visuales, cada uno diseñado para responder a uno o más de los requerimientos clave del caso de estudio:

Análisis de las Visualizaciones del Dashboard

El dashboard (mostrado en la imagen) está compuesto por 7 objetos visuales, cada uno diseñado para responder a uno o más de los requerimientos clave del caso de estudio:

1. **Mapa - Ingresos por País (Req. 1)**
   - **Qué es:** Un objeto de mapa que muestra la `Sum(ingresos_totales)` (Suma de ingresos) por `pais_sede_desc` (País). La intensidad del color indica el total de ingresos.
   - **Responde a:** Requerimiento 1: "¿Qué países generan los mayores ingresos por venta de boletos...?".
2. **Gráfica de Barras - Películas por Sección (Req. 1)**
   - **Qué es:** Una gráfica de barras que muestra el `Count(pelicula_id)` (Conteo de películas) por `nombre_seccion` (ej. Comedia, Drama, etc.).
   - **Responde a:** La segunda parte del Requerimiento 1: "...y cuántas películas presentaron en cada sección?".
3. **Gráfica de Barras - Ventas por Lugar (Req. 2 y 4)**
   - **Qué es:** Una gráfica de barras que muestra la `Sum(ingresos_totales)` (Suma de ingresos) por `lugar_cine_desc` (el cine o teatro).
   - **Responde a:** Requerimiento 2 ("¿Qué eventos registraron las mayores ventas por lugar...?") y Requerimiento 4 ("¿Qué lugares (salas/teatros) acumulan las mayores ganancias...?"). Nota: Para responder exactamente al Req. 4, la métrica se puede cambiar a `Sum(ganancias_totales)`.
4. **Gráfica de Líneas - Picos de Venta Diarios (Req. 2 y 4)**
   - **Qué es:** Una gráfica de líneas que muestra la `Sum(ingresos_totales)` (Suma de ingresos) a lo largo de los `dia_descripcion` (días de la semana).
   - **Responde a:** Requerimiento 2 ("...y en qué fechas ocurrieron esos picos de venta?") y Requerimiento 4 ("...y cómo han evolucionado sus ventas por período (dia...)?").
5. **Gráfica de Pastel - Comparativa de Eventos (Req. 3)**
   - **Qué es:** Una gráfica de pastel que muestra la proporción de ventas por `nombre_evento` (ej. Gala Beneficencia, Premier Latinoamericana, Función Regular).
   - **Responde a:** Requerimiento 3, permitiendo la comparación visual de ventas entre "Premier Universal" y "Premier Latinoamericana" y el resto de los eventos.
6. **Gráfica de Barras - Ganancias por Sección (Req. 6)**
   - **Qué es:** Una gráfica de barras que muestra la `Sum(ganancias_totales)` (Suma de ganancias) por `nombre_seccion`.
   - **Responde a:** Esta es la respuesta directa a la parte más importante del Requerimiento 6: "...y qué secciones muestran mejor rentabilidad?". Permite identificar al instante las secciones más rentables (como "Comedia" o "Documental") en lugar de solo el volumen de ventas.
7. **Tabla - Detalle de Películas (Req. 5)**
   - **Qué es:** Una tabla detallada que muestra las dimensiones `titulo_pelicula`, `pais_sede_desc` y `nombre_seccion`.
   - **Métrica:** `Sum(boletos_vendidos)` (Suma de boletos vendidos).

- **Responde a:** Requerimiento 5: "¿Qué películas son las más vendidos y en qué países, secciones y lugares concentran sus mejores resultados?". El usuario puede ordenar la tabla por "`Sum(boletos_vendidos)`" para ver el ranking y, gracias al modelo asociativo, filtrar por país o sección para un análisis más profundo.